

AI for managing technical debt

About me

- Director of Koddian software engineering at Kumojin
- Software craftsman
- 15 years of experience
- Advanced AI user for development for 18 months



The problem: technical debt accumulates

- External ecosystem degradation
- Accumulation of internal decisions
- Growth complexity

AI can help... or make it worse

Potential

- Analyze thousands of lines
- Detect obsolete patterns
- Propose migrations
- Automate refactoring

Risks

- Out-of-context suggestions
- Unrealistic estimates
- Untested changes
- **Technical debt increases instead of decreasing**

Definitions: the tools

LLMs (Large Language Models)

Advanced generative AI systems that understand and generate natural language.

Examples: GPT, Claude, Gemini

Coding agent

A tool that reads, modifies, or fixes code autonomously using natural language as input.

Examples: Codex, Claude Code, Cursor

How does an LLM work?

Three major steps to create an LLM:

1. Pre-training

The model learns to predict the next token from billions of text examples.

💰 Millions of \$ · ⌚ Weeks/months · 🖥️ Thousands of GPUs

2. Supervised fine-tuning (SFT)

The model learns to follow instructions from human-annotated examples.

📊 Less data · 👤 Curated examples · 🎯 Instruction following

3. Reinforcement learning from human feedback (RLHF)

The model learns to be helpful, honest, and harmless through human preferences.

👥 Human feedback · 🏆 Reward model · ✅ Alignment

What happens when you send a prompt?

1. Your prompt is **tokenized** (split into chunks)

```
"Analyze this function" → ["Analyze", "this", "function"]
```

2. The model processes these tokens through its neural network
3. **Predicts the MOST LIKELY next token** based on learned patterns

```
"Analyze this" → next token: probably "function"
```

4. Adds this token to the sequence
5. Repeats until generating a complete response

⚠ **Key point:** "Most likely" ≠ "Correct"

This is **statistical prediction**, not magic.

Key capabilities

✓ Massive analysis

- Thousands of lines in seconds

✓ Pattern recognition

- Anti-patterns, code smells

✓ Learning by example

- Give it examples, it adapts

✓ Tool connection

- Read files, execute code

✓ Code generation

- Boilerplate, tests, guided refactors

Limitations

⚠ Knowledge cutoff (varies by model)

- Doesn't know recent library versions for example

⚠ Hallucinations

- Can invent APIs

⚠ Context window (varies by model)

- Memory limit

⚠ Complex reasoning

- Algorithms, advanced math

⚠ Non-determinism

- Same prompt → different responses

The philosophy

Two approaches to using AI in development

Vibe Coding

Prompts	"Create a user profile page"
Validation	No review
Understanding	Blind copy-paste
Speed	 Fast at first
Long term	 Technical debt

Mindset: "It works, that's good enough"

AI-assisted engineering

Prompts	"Refactor UserService to use DI"
Validation	Review + tests at each step
Understanding	Understand and adapt
Speed	 Slower at first
Long term	 Maintainable

Mindset: "AI proposes, I validate"

Objectives of this presentation

What you'll see

-  Practice **AI-assisted engineering** (not vibe coding)
-  Understand and **validate each step**
-  Apply on **real legacy code**

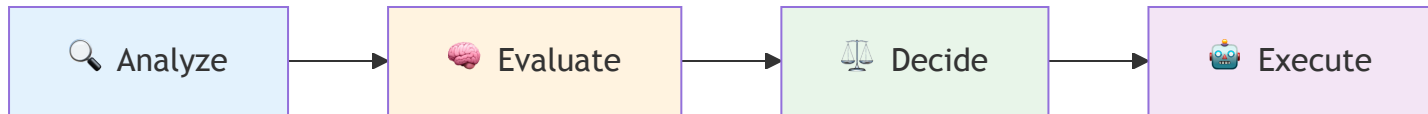
Our test subject: extreme-carpaccio

A kata that illustrates a **common professional situation**:

- Inheriting a project that hasn't been maintained for several years.
- Despite its initial quality, the code now has significant technical debt.

Positive point: an existing test suite

Demo roadmap



The pattern to observe

AI proposes → Human validates → AI executes

Why this pattern?

- Leverages AI speed without sacrificing control
- Reduces cognitive load: AI explores, you decide
- Scalable: adaptable from individual code to complex systems

Code analysis

Technical debt analysis prompt

Analyze this codebase for technical debt and architectural issues.

Focus on:

- Code smells and anti-patterns
- Outdated dependencies and framework versions
- Architectural problems (coupling, separation of concerns, etc.)
- Security vulnerabilities from deprecated packages
- Performance bottlenecks or inefficient patterns
- Missing error handling
- Inconsistent code patterns across the codebase

For each issue you identify:

1. Specify the exact location (file and line numbers)
2. Explain why it's problematic
3. Rate the severity (critical, high, medium, low)
4. Estimate effort to fix (hours or story points)
5. Suggest a specific remediation approach

After the analysis, generate a prioritized action plan with:

- Quick wins (high value, low effort)
- Critical issues that block modernization
- Long-term refactoring goals

Code analysis - Results

What the AI found

- **Outdated dependencies:** React 0.12, jasmine-node
- **CVE vulnerabilities:** known security flaws
- **Anti-patterns:** callback hell, duplicated code
- **Clear structure:** exact location + severity + effort + solution

What needs validation

- **Business context:** "arbitrary code execution" → yes, it's a feature
- **Estimates:** "TypeScript migration: 8-10 days" → indicator, not truth
- **Knowledge cutoff:** doesn't always know the latest versions
- **Priorities:** the AI doesn't know your project constraints

Major migration: React 19.2

Migration prompt

Plan the React migration from the frontend to the latest version (19.2).

Constraints:

- Do not migrate to TypeScript
- Ensure libraries compatibility with React 19.2
- Propose E2E testing strategy to validate before and after

Major migration: React 19.2

Interactive approach

- Claude analyzes and asks questions
- You guide the strategy
- Claude generates a detailed plan

Compare before migrating

The agent analyzes options and recommends the best approach for your context.

Example prompt

Search body-parser alternatives.

Compare:

- Performance and bundle size
- API compatibility
- Migration effort
- Active maintenance

Recommend the best option for our use case.

Remove rather than migrate

The best dependency is the one you don't have. The agent identifies what can return to native APIs.

Example prompt

```
Analyze our use of lodash in the codebase.
```

```
Identify functions that can be replaced with native JavaScript APIs.
```

```
For each replacement:
```

- Show the native equivalent
- Confirm identical behavior
- Highlight any edge cases
- Estimate migration effort

```
Prioritize high-usage functions first.
```

Automate with a custom codemod

The AI looks for an existing codemod, and if none fits, writes one with dry-run and human validation.

Example prompt

```
/codemod
```

```
Migrate our jasmine-node test suite to jest spy semantics:
```

- spyOn(x, y).andReturn(z) → jest.spyOn(x, y).mockReturnValue(z)
- spyOn(x, y).andCallFake(fn) → jest.spyOn(x, y).mockImplementation(fn)
- jasmine.any(Type) → expect.any(Type)

40+ occurrences across 4 spec files. No public codemod exists for this transformation.

Tool: Codemod MCP

```
npx codemod ai
```

And in real life?

- Replace React-Scripts with Vite
- Migrate from React class components to function components
- Migrate React-Router 5x to 7x
- Major Material UI migration
- Automate dependency updates
- Support for audits

What's next? Pay the debt continuously

So far: a developer paying down the debt reactively. What's next: agents paying it down continuously.

The drift problem

Agents reproduce existing patterns, including bad ones. The code slowly drifts.

At OpenAI, **20% of the week (every Friday) cleaning up the "AI slop"** before they automated it.

Source: *Harness engineering*, Ryan Lopopolo, OpenAI (Feb. 2026).

gh-aw – the open source version

`github/gh-aw`: agentic workflows in GitHub Actions, declared in markdown.

```
gh extension install github/gh-aw
```

The answer: the harness

1. Encode the "golden principles" in the repo
2. Background agents that scan for drift
3. Small, targeted, automerge cleanup PRs
4. **Pay down the debt continuously** rather than in big periodic projects

Formalize what the agent must know

Without context, the agent replicates what it sees in the code, including bad practices.

AGENTS.md

Always loaded into the context.

```
# Conventions

- Tests: vitest, not jasmine
- Logger: pino, never console.*
- No moment.js
```

Skills

The agent reads descriptions in context and loads the full content when the task matches.

- **Auto:** writing-tests , debugging , pr-content ...
- **Explicit:** /code-review , /codemod , /release

Tool: rulesync

Single source → configs for **25+ tools**: Claude Code, Cursor, Copilot, Gemini CLI, Codex, Cline... Rules, skills, commands, MCP, permissions.

```
npx rulesync generate
```

Key takeaways

Five demos, one approach

1. **Analyze**: the AI explores, we validate the findings
2. **Plan**: the AI proposes a plan, we steer it
3. **Decide**: the AI compares the alternatives, we choose
4. **Simplify**: the AI spots unnecessary dependencies, we measure the effort
5. **Automate**: the AI writes the codemod, we validate the diff

Where to start on your projects

- **Start small**: code analysis (low risk)
- **Develop your critical thinking**: question everything
- **Expand progressively**: migration → refactoring → automation
- **Keep control**: AI proposes, you decide

Questions?

<https://github.com/xballoy/presentation-ai-tech-debt>